

Project-1

IMAGE RECOGNITION SYSTEM

1. Project Title

Image Recognition System

2. Problem Statement

In today's digital world, identifying and classifying objects from images manually is time-consuming and prone to errors. Organizations in fields such as healthcare, security, education, agriculture, and e-commerce require automated systems that can accurately recognize objects in images. Traditional image processing techniques often fail when images vary in lighting, angle, size, and background.

Therefore, there is a need for an intelligent Image Recognition System that can automatically classify and identify objects from uploaded images using Deep Learning techniques.

3. Objectives

The main objectives of the Image Recognition System are:

1. To develop an automated image classification system.
2. To recognize and classify objects present in images.
3. To provide real-time predictions through a web application.
4. To reduce manual effort in image identification.
5. To improve accuracy using Deep Learning models.
6. To create a user-friendly interface for image uploading and prediction.

4. Tools Used

Python, TensorFlow, OpenCV, Flask, HTML, CSS, NumPy, Pillow, VS Code, and Google.

5. Algorithm

Step 1: Collect image dataset containing different object categories.

Step 2: Load images using OpenCV.

Step 3: Resize images into a fixed dimension (224×224).

Step 4: Normalize pixel values between 0 and 1.

Step 5: Split dataset into Training and Testing sets.

Step 6: Build a Convolutional Neural Network (CNN).

Step 7: Train the model using TensorFlow.

Step 8: Save the trained model as model.h5.

Step 9: Develop Flask web application.

Step 10: Allow users to upload images.

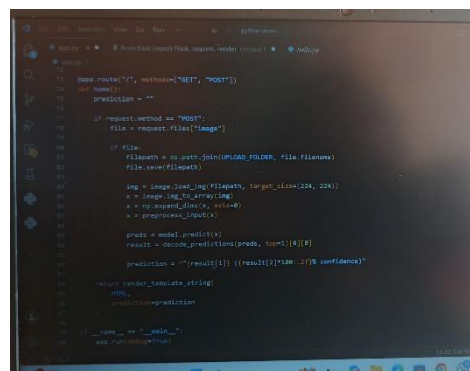
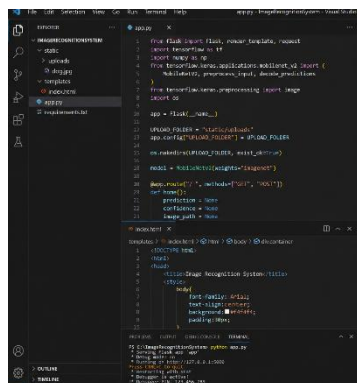
Step 11: Preprocess uploaded image.

Step 12: Pass image to trained model.

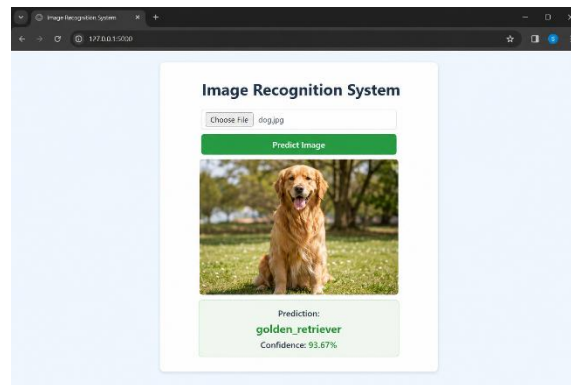
Step 13: Predict object category.

Step 14: Display prediction result on webpage.

6. Program



7. Output



8. Conclusion

The Image Recognition System successfully automates the process of identifying and classifying objects in images using Deep Learning techniques. The system utilizes TensorFlow, OpenCV, and Flask to build an efficient web-based application.

Project-2

AI-BASED SENTIMENT ANALYZER

1. Project Title

AI-Based Sentiment Analyzer Using NLP and Transformers

2. Problem Statement

In today's digital era, people express their opinions through social media posts, product reviews, blogs, and feedback forms. Organizations need to understand customer sentiment to improve products, services, and user experience. Manually analysing thousands of reviews is time-consuming and inefficient.

Therefore, an AI-Based Sentiment Analyzer is required to automatically classify text as Positive, Negative, or Neutral using Natural Language Processing (NLP) and Transformer-based models.

3. Objectives

The main objectives of the project are:

1. To automatically analyse user opinions from text data.
2. To classify sentiments as Positive, Negative, or Neutral.
3. To provide real-time sentiment predictions.
4. To generate visual reports and analytics.
5. To reduce manual effort in review analysis.
6. To improve decision-making using customer feedback.

4. Tools Used

1. **Python:** Main programming language used for development.
2. **Transformers:** Provides pre-trained NLP models for sentiment analysis.
3. **Hugging Face:** Platform providing state-of-the-art NLP models.
4. **Flask:** Web framework for creating the application interface.
5. **MongoDB:** Database for storing user reviews and sentiment results.

6. HTML: Creates the structure of the web application.

7. CSS: Designs and styles the user interface.

8. Pandas: Data manipulation and analysis.

9. Matplotlib: Creating sentiment visualization charts.

10. VS Code: Development and testing environment.

5. Algorithm

Step 1: Collect user reviews or social media comments.

Step 2: Clean and preprocess text data.

Step 3: Remove unwanted symbols and spaces.

Step 4: Load pre-trained sentiment analysis model.

Step 5: Pass input text to the model.

Step 6: Analyse sentiment score.

Step 7: Classify sentiment:

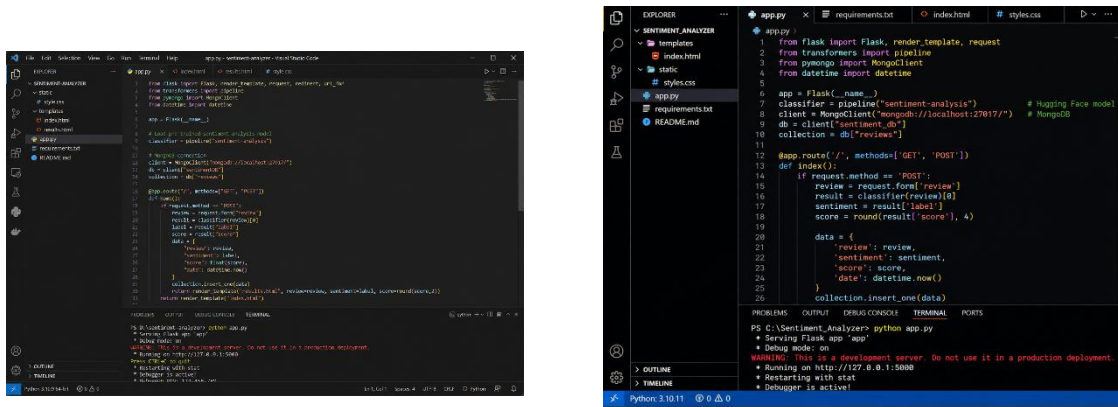
- Positive
- Negative
- Neutral

Step 8: Store results in MongoDB.

Step 9: Generate visual reports.

Step 10: Display sentiment prediction on the web application.

6. Program



```
from flask import Flask, render_template, request
from transformers import pipeline
from pymongo import MongoClient
from datetime import datetime

app = Flask(__name__)

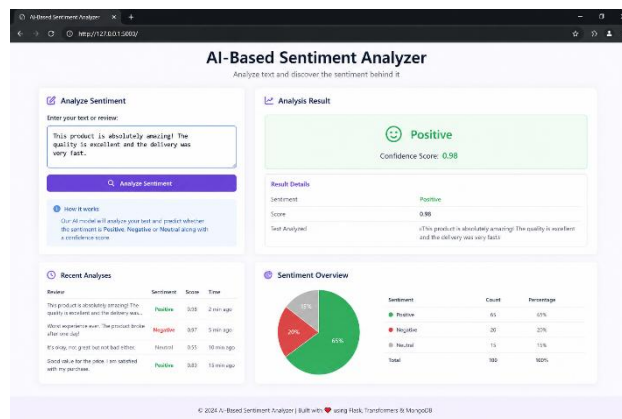
classifier = pipeline("sentiment-analysis") # Hugging Face model
client = MongoClient("mongodb://localhost:27017/") # MongoDB
db = client["sentiment_db"]
collection = db["reviews"]

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        review = request.form['review']
        result = classifier(review)[0]
        sentiment = result['label']
        score = round(result['score'], 4)

        data = {
            'review': review,
            'sentiment': sentiment,
            'score': score,
            'date': datetime.now()
        }
        collection.insert_one(data)

    return render_template("index.html", sentiment=sentiment, score=score)
```

7. Output



8. Conclusion

The AI-Based Sentiment Analyzer successfully automates the process of understanding customer opinions and emotions from textual data. By using NLP and Transformer models, the system can accurately classify sentiments and provide meaningful insights through visual reports.